Practical 7: Implementation of Round Robin Scheduling

Author: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: Feburary 25, 2026

Topic: Implementation of scheduling algo: Round Robin

1. Theory: Round Robin (RR)

Round Robin (RR) is a preemptive CPU scheduling algorithm where each process is assigned a fixed time slot (quantum) in a cyclic order. If a process does not finish in its time quantum, it is placed at the end of the queue.

- **Preemptive**: Yes (always switches after quantum expires)
- **Parameter**: Time Quantum (Q)
- **Fairness**: Ensures all processes get CPU time regularly.
- **Context Switches**: Frequent, may increase overhead.
- **Used In**: Time-sharing systems, multitasking OS.

Key Points

- Each process waits its turn in the ready queue.
- If a process finishes before quantum, CPU switches immediately.
- If not, process is moved to the end of the queue.

.

2. Example Scenario (Solved)

Process	Arrival Time	Burst Time
P1	0	5
P2	1	4
P3	2	2
P4	3	1

- **Time Quantum:** 2

Gantt Chart (Step-by-Step)

Time	Process	Remaining Burst
0-2	P1	3
2-4	P2	2
4-6	P3	0 (finishes)
6-7	P4	0 (finishes)
7-9	P1	1
9-11	P2	0 (finishes)
11-12	P1	0 (finishes)

Completion Times

- P3: 6

- P4: 7
- P2: 11
- P1: 12

Waiting Times

- P1: (Start at 0, runs at 0-2, 7-9, 11-12) = (0) + (7-2=5) + (11-9=2) = 0+5+2 = 7
- P2: (Start at 2, runs at 2-4, 9-11) = (2-1=1) + (9-4=5) = 1+5 = 6
- P3: (Start at 4, runs at 4-6) = (4-2=2) = 2
- P4: (Start at 6, runs at 6-7) = (6-3=3) = 3

Turnaround Times

- P1: 12 - 0 = 12
- P2: 11 - 1 = 10
- P3: 6 - 2 = 4
- P4: 7 - 3 = 4

Final Table

Process	Arrival	Burst	Waiting	Turnaround
P1	0	5	7	12
P2	1	4	6	10
P3	2	2	2	4
P4	3	1	3	4

.

3. C Implementation

```c
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, remaining, waiting, turnaround;
};

void inputProcesses(struct Process p[], int n) {
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival: ", i + 1);
    scanf("%d", &p[i].arrival);
    printf("P%d Burst: ", i + 1);
    scanf("%d", &p[i].burst);
    p[i].remaining = p[i].burst;
  }
}

void calculateRoundRobin(struct Process p[], int n, int quantum) {
  int time = 0, completed = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
```

```
      if (p[i].remaining > 0 && p[i].arrival <= time) {
        found = 1;
        int exec = (p[i].remaining < quantum) ? p[i].remaining : quantum;
        printf("P%d\t\t%d\t%d\n", p[i].pid, time, time + exec);
        p[i].remaining -= exec;
        time += exec;
        if (p[i].remaining == 0) {
          p[i].turnaround = time - p[i].arrival;
          p[i].waiting = p[i].turnaround - p[i].burst;
          completed++;
        }
      }
    }
    if (!found) time++;
  }
}

void printTable(struct Process p[], int n) {
  float avg_wt = 0, avg_tat = 0;
  printf("\nPID\tAT\tBT\tWT\tTAT\n");
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

int main() {
  struct Process p[MAX];
  int n, q;
  printf("Round Robin\nEnter number of processes: ");
  scanf("%d", &n);
  inputProcesses(p, n);
  printf("Enter Time Quantum: ");
  scanf("%d", &q);
  calculateRoundRobin(p, n, q);
  printTable(p, n);
  return 0;
}
```

.

4. Output Screenshots

- **Round Robin Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr7
❯ ./a.out
Round Robin
Enter number of processes: 5
P1 Arrival: 2
P1 Burst: 4
P2 Arrival: 2
P2 Burst: 3
P3 Arrival: 1
P3 Burst: 6
P4 Arrival: 3
P4 Burst: 1
P5 Arrival: 2
P5 Burst: 5
Enter Time Quantum: 2

Order of Execution:
process_name    start   end
P3              1       3
P4              3       4
P5              4       6
P1              6       8
P2              8       10
P3              10      12
P5              12      14
P1              14      16
P2              16      17
P3              17      19
P5              19      20

PID     AT      BT      WT      TAT
P1      2       4       10      14
P2      2       3       12      15
P3      1       6       12      18
P4      3       1       0       1
P5      2       5       13      18

Average Waiting Time: 9.40
Average Turnaround Time: 13.20
```